CASE STUDY · 2025

Plant Care

An AI-powered plant identification and care companion built as a PWA — from scan to personalised permaculture wisdom, all in one tap.

Next.js 16 App Router | Supabase Auth + DB | Google Gemini Vision | PlantNet API | Groq LLM | Framer Motion | Material 3 | PWA

OVERVIEW

What is Plant Care?

Plant Care is a mobile-first Progressive Web App that lets anyone photograph a plant, instantly identify it using PlantNet's species database cross-referenced with Gemini Vision, and receive tailored care instructions. The app tracks your entire plant collection, schedules watering reminders, monitors plant health, and surfaces personalised permaculture tips based on the plants you actually own.

Built solo in under two weeks as a personal project turned product experiment. The entire stack — auth, AI, database, UI — is serverless and deploys to Vercel in a single `git push`.

3	12	<2s	PWA
AI APIS INTEGRATED	PERMACULTURE PRINCIPLES	PLANT ID LATENCY	INSTALLABLE, OFFLINE-READY

PROBLEM

Why another plant app?

Existing plant apps either charge $9.99/month for species ID (PlantSnap, PictureThis) or give you generic care sheets that ignore your specific environment — light levels, watering schedule, companion planting. There's nothing that *learns your collection* and surfaces wisdom that's actually relevant to the plants on your windowsill.

💸

Expensive gatekeeping

Most AI plant ID sits behind $8–15/month subscriptions for what's essentially one API call.

📋

Generic care advice

Care sheets are the same for every Monstera regardless of your window direction, watering frequency, or companion plants.

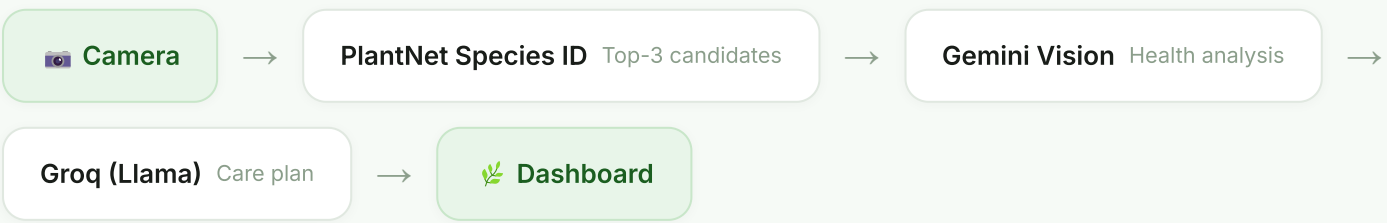
🔔

No intelligent scheduling

Static "water every 7 days" ignores light level, season, and actual plant stress signals the AI already detected.

SOLUTION

How it works



The scan pipeline runs three AI models in sequence: PlantNet narrows the species using a botanical visual model, Gemini Vision describes the specific plant's health and stress signs, and Groq synthesises personalised care instructions tuned to the identified species, light level, and existing watering schedule. Results are stored in Supabase and surfaced on the dashboard.

AI Plant Identification

PlantNet returns top-3 species candidates with confidence scores. Gemini cross-validates with visual health analysis.

Adaptive Care Engine

Watering intervals adjust dynamically based on light level (low light = longer interval), season, and last-watered date.

Stacked Tip Deck

Permaculture tips rendered as a swipe-to-dismiss card stack. Most relevant tips (personalised to your plants) always surface first.

Watering Urgency

Plants overdue for water surface in an alert strip with quick-water button that logs the action and resets the schedule in one tap.

Passwordless Auth

Supabase OTP magic link + Google OAuth with PKCE flow. No passwords stored. New users land in the app in under 30 seconds.

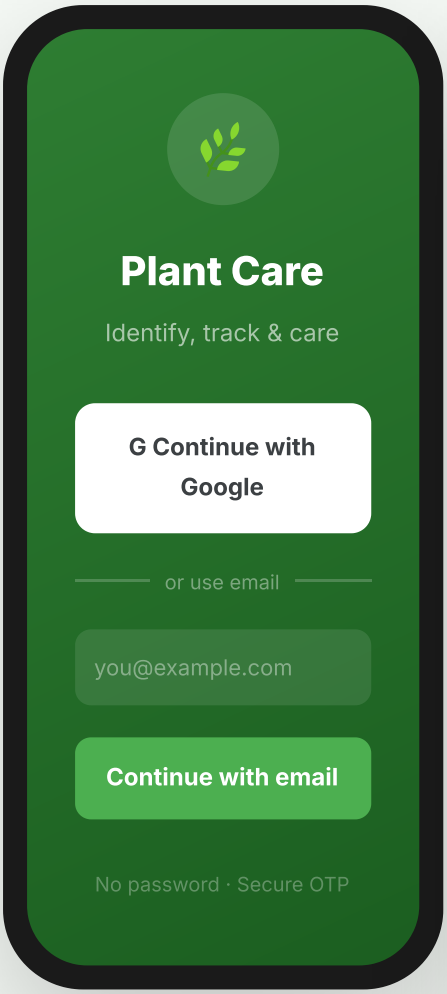
PWA + Installable

Manifest + service worker make it installable on iOS and Android. Web push notifications planned for watering reminders.

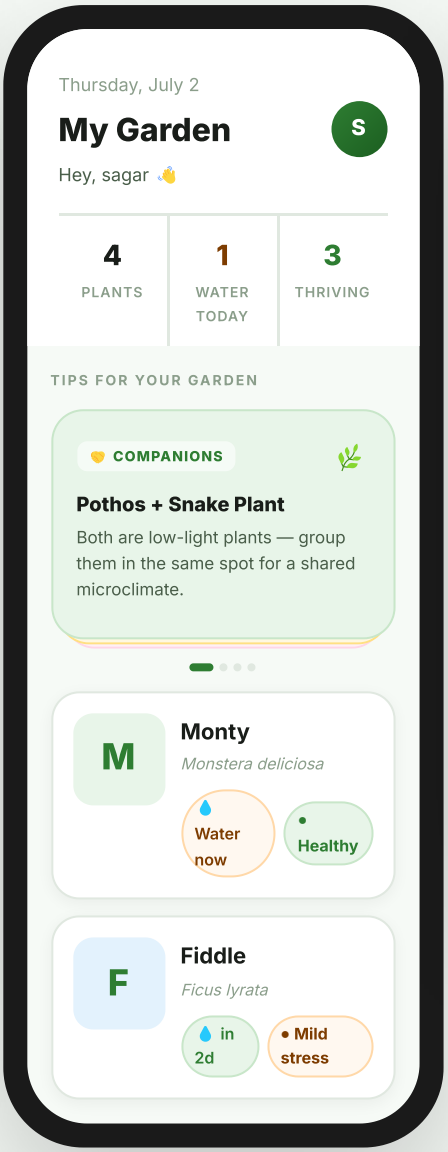
SCREENS

Interface

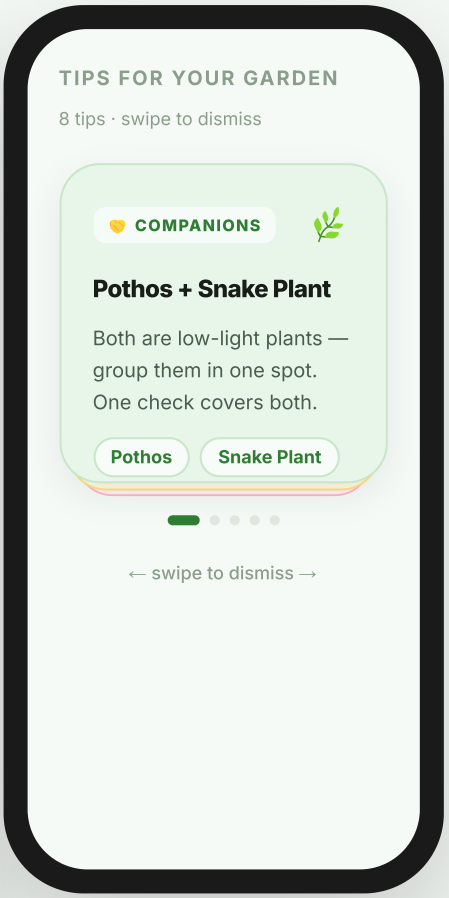
The entire UI is custom-built with inline styles referencing a shared M3-aligned token system — no component library. This gave full control over every motion and shadow.



SIGN IN



DASHBOARD



STACKED TIPS

Live app: plant-vision-three.vercel.app — sign in with Google or email OTP to see your own personalised dashboard.

DESIGN DECISIONS

Key choices & trade-offs

1 Stacked card deck instead of horizontal carousel

Horizontal scroll carousels hide content depth — users don't know how many tips exist. A stacked deck communicates "there's more below" visually and makes the interaction tactile (swipe to dismiss, like Tinder). The most relevant personalized tip is always on top. Progress dots show position without needing a scroll indicator.

2

Framer Motion spring physics over CSS transitions

CSS `transition` uses fixed easing curves. Spring physics (stiffness: 360, damping: 38) creates motion that responds to gesture velocity — a fast swipe produces a snappier exit, a slow drag produces a gentler one. This makes the interface feel alive rather than mechanical. Used for card stack, plant card stagger-in, stat count animations, and badge transitions.

3

Three AI APIs instead of one

PlantNet's botanical visual model is best-in-class for species ID but gives no care information. Gemini is excellent at visual health analysis but not fine-tuned for species. Groq (Llama 3) is extremely fast (100+ tok/s) for text synthesis. Chaining all three keeps latency under 2 seconds while maximising each model's strength.

4

Passwordless-first auth

Email OTP magic links + Google OAuth via Supabase PKCE flow. No passwords means no reset flows, no brute force risk, and faster sign-up for casual users who just want to scan a plant. The PKCE callback route at `/auth/callback` exchanges the auth code server-side for security.

5

Inline styles with a shared token object instead of Tailwind classes

The M3-aligned `T` token object (colors, shadows, border-radii) is the single source of truth. Changing `T.green` updates every use site instantly. No class name collisions, no purge configuration, and the token values are visible and refactorable in code rather than scattered across className strings. Trade-off: no Tailwind's responsive utilities — but this is a mobile-first app with a fixed max-width, so that matters less.

6

Personalised permaculture tips computed at render time

`getPersonalizedTips(plants)` runs in the browser via `useMemo` — no server round trip. It matches the user's actual plant names against companion planting data, pairs plants sharing light levels, identifies drought-tolerant plants for batch-watering tips, and falls back to general permaculture principles. The "most relevant first" ordering naturally places user-specific tips before generic ones.

STACK

Technology choices

Next.js 16
App Router, server routes, PWA

Supabase
Postgres, Auth, Row-level security

PlantNet API
Species identification (Top-3)

Google Gemini
Vision — health + stress analysis

Groq (Llama 3)
Care plan synthesis, 100+ tok/s

Framer Motion 12
Spring physics, AnimatePresence

Material 3 tokens
Design system, shape + colour

Vercel
Hosting, edge functions, CI/CD

Google OAuth (PKCE)
Social sign-in, no passwords

TypeScript
Full type safety end to end

IMPLEMENTATION DETAIL

Stacked card swipe pattern

The tip deck uses `useMotionValue` + `useTransform` for live drag feedback and `AnimatePresence` for spring-based card transitions. Three cards are always in the DOM — only the top card is draggable.

```
// Each card tracks its own drag x-position
const x = useMotionValue(0);
const rotate = useTransform(x, [-200, 200], [-15, 15]);
const opacity = useTransform(x, [-160, -80, 0, 80, 160], [0, 1, 1, 1, 0]);

// Stack position via animate prop — springs to new value on index change
animate={{ scale: 1 - stackIndex * 0.048, y: stackIndex * 12, opacity: 1 }}

// Dismiss threshold: 80px offset → remove top card, next springs forward
onDragEnd={(_, info) => {
  if (Math.abs(info.offset.x) > 80) onDismiss();
```

```
}}

// Exit: card flies off-screen with rotation
exit={{ x: 380, rotate: 24, opacity: 0, transition: { duration: 0.28 } }}
```

The animated progress dots are a single `motion.div` with an `animate={{ width }}` that expands the active dot to 20px. React state drives which index is active — no separate dot component needed.

REFLECTION

What I learned

AI pipeline orchestration

Chaining three AI APIs with different response shapes requires defensive parsing at each stage. PlantNet confidence scores gate whether Gemini runs; Gemini's output shapes the Groq prompt.

OAuth PKCE complexity

Google OAuth with Supabase's SSR package requires a dedicated server-side callback route for the code exchange. The redirect allowlist in Supabase must exactly match the production URL — a common misconfiguration.

Framer Motion + React 19

Framer Motion 12 works cleanly with React 19's concurrent renderer. The key: never mix `animate` targets with `style` MotionValues for the same property on the same element.

Permaculture as UX metaphor

Mapping permaculture's 12 principles to houseplant care advice was a creative constraint that forced unusually specific and memorable tips — much more engaging than generic "water when topsoil is dry" copy.

Plant Care

Built by **Sagar** · 2025 · **plant-vision-three.vercel.app**

Next.js · Supabase · PlantNet · Gemini · Groq · Framer Motion · Vercel